Software Architecture Document

Voice Assistant for Multimedia Controls

Team 3 | Version 1.3 | 11 November 2026

Architecture baseline: updated SRS version 1.3 and VoiceAssistant_Project_Complete_Guide.txt. Prepared for Dhanush S S and Team 3. Instructor review of the changed prototype scope remains pending. This document defines the intended architecture and distinguishes guide-described behavior from independently verified execution.

1 Purpose and architectural scope

The prototype controls multimedia on a Windows laptop using speech captured by an INMP441 microphone connected to an ESP32-WROOM. The ESP32 acquires audio and streams it over USB serial. Python on Windows creates a WAV file, runs local Faster-Whisper transcription, classifies a keyword, maps an action and dispatches a media key through PyAutoGUI. Each command requires a new invocation. The laptop is essential to recognition and execution.

Cloud speech recognition is excluded. Initial model installation may download files; inference must work locally after caching. Network access used by a media streaming application is outside the recognition path. Wake word, continuous listening, on-board tinyML, BLE HID, physical mute, Wi-Fi provisioning, factory reset, battery listening and OTA are outside the current prototype baseline. Earlier modules and tests are preserved as historical work.

2 Team ownership and review

Member and SRN	Architecture responsibility and files	Independent cross-tester
Dhanush S S — PES2UG24CS154	Integration Lead and Project Manager. main.py, CLI port configuration, orchestration, repository integration, release evidence and SRS coordination.	G S Dheekshith
G S Dheekshith — PES2UG24CS166	Audio Module Lead. VoiceAssistant_ESP32.ino, receive_audio.py, microphone wiring, I2S conversion, bounded capture/streaming, framing and WAV creation.	Harsh Mannan
Harsh Mannan — PES2UG24CS183	Recognition Module Lead. speech_to_text.py, command_classifier.py, model configuration, transcript normalization, keyword behavior and accuracy evaluation. Coordinates SAD/STP reconciliation.	Chiranthan Shankar
Chiranthan Shankar — PES2UG24CS137	Output and Maintenance Lead. command_mapper.py, media_controller.py, intent/action/key mapping, execution outcomes and diagnostics.	Dhanush S S

Assignments identify responsibility rather than historical authorship. All members contribute their module design and tests. Dhanush assembles integration changes; module owners raise their own PRs. Independent testers verify another member's module. Joint system validation includes all four members.

3 Architectural drivers

The main drivers are bounded ESP32 memory, a reliable serial protocol, local recognition, separation of concerns, predictable command mapping and visible error outcomes. Firmware must avoid a complete five-second global audio buffer. Desktop stages communicate through public functions; mapping must not directly read audio or call the recognizer. The execution adapter hides operating-system calls from classification.

A fixed five-second capture cannot meet the earlier three-second wake-to-action target. PLAY and PAUSE dispatch the same playpause toggle, so dispatch success does not guarantee the requested playback state. A Windows media key can be handled differently by different applications. These limits shape acceptance tests and must be visible during evaluation.

4 System views

4.1 Context view

User -> microphone -> ESP32 acquisition device -> USB serial -> Windows Python application -> Windows media subsystem -> active player.

The user starts a run using the COM port. The player is an external system; the prototype does not implement playback or a streaming-service API. Arduino IDE is a deployment tool, not part of the runtime command path.

4.2 Logical component view

Layer	Component	Responsibility
Acquisition	INMP441 and firmware	Sample I2S audio and convert signed 32-bit input slots to the configured PCM16 representation.
Transport and recording	receive_audio.py	Request one recording, reconstruct a complete validated binary response and write a WAV.
Recognition	speech_to_text.py	Load/use local Faster-Whisper base on CPU with int8 and beam size 1; join transcript segments.
Interpretation	command_classifier.py	Lowercase, remove punctuation, split words and return an intent or UNKNOWN.
Mapping	command_mapper.py	Translate intent into a stable action identifier or unknown.
Execution	media_controller.py	Translate valid actions into PyAutoGUI media-key calls.
Coordination	main.py	Validate CLI usage, sequence stages, expose outcomes and terminate safely on errors.

4.3 Runtime sequence

1. User runs python main.py COM3 using the actual port; Serial Monitor is closed.

2. Application makes the model ready according to the implementation's loading strategy. Model-load failure must prevent dispatch.

3. Receiver opens the port, allows for board readiness/reset and clears stale input as defined by the receiver implementation.

4. Receiver sends ASCII r; firmware captures five seconds and streams the framed PCM response.

5. Receiver validates header, declared length, payload and trailer before returning a WAV path.

6. Recognizer transcribes the WAV locally and joins segments into text.

7. Classifier normalizes the text and identifies a keyword or UNKNOWN.

8. Mapper selects an action. UNKNOWN or invalid input terminates without key dispatch.

9. Controller sends the configured key and reports dispatch outcome. User/tester observes the player separately.

10. Resources are released and the run exits. Another command requires another invocation.

The guide's presentation order and demo may show model loading before recording. main.py's actual loading behavior must be recorded when measuring timings; loading order is not assumed to be a guaranteed API contract.

4.4 Deployment view

The ESP32 runs Arduino firmware. The Windows laptop runs Python in a virtual environment and holds the cached model and runtime WAV. USB supplies the serial link and normally board power. A Windows media application receives host media events. No cloud endpoint is in the recognition architecture. The prototype does not require the ESP32 Bluetooth radio or Wi-Fi connection for command execution.

5 Interfaces and data contracts

5.1 Hardware configuration

INMP441 pin	ESP32 signal
VDD	3V3
GND	GND
SCK or BCLK	GPIO 26
WS or LRCL	GPIO 25
SD	GPIO 33
L/R	GND for left channel

Verify the physical board pinout before powering. I2S input slots are 32 bits; output is signed PCM16, mono, 16 kHz. Exact conversion shifts/channel settings are checked against the submitted firmware rather than invented here.

5.2 Binary serial contract

Field	Encoding and size
Baud	921600
Request	ASCII r, one byte
Header marker	ASCII AUD0, four bytes
Payload length	Unsigned little-endian 32-bit integer, four bytes
PCM payload	160,000 bytes for five seconds of PCM16 mono at 16 kHz
Trailer	ASCII END0, four bytes

The response is 160,012 bytes; the request is separate. No newline or text log is permitted inside the packet. Serial read calls may return fewer bytes than requested; read_exact must accumulate until complete or fail under a finite documented timeout. Unexpected payload lengths must be rejected before unbounded allocation or waiting. Packet readiness delay, read timeout and resynchronization policy need approved implementation values. No checksum, acknowledgement, sequence number or transport retry is claimed by this contract.

5.3 Public module interfaces

Interface	Behavioral contract
record_from_esp32(port)	Given a selected port, return the path to a validated mono PCM16 WAV or report acquisition failure.
transcribe_audio(audio_file)	Given a WAV, return joined transcript text or report model/audio/inference failure.
classify_command(transcript)	Return PLAY, PAUSE, NEXT, PREVIOUS, MUTE, VOLUME_UP, VOLUME_DOWN or UNKNOWN according to the current keyword rules.
map_command(intent)	Return the documented action identifier; unknown intent maps to unknown.
execute_action(action)	Dispatch a valid configured Windows key; invalid actions produce no dispatch. Report dispatch failure honestly.

Concrete exception classes, return types and shared error representation remain subject to code reconciliation. main.py must not continue to execution after an upstream failure. Unit tests inject serial streams, model results and dispatch adapters instead of requiring devices.

5.4 Command mapping

Keyword	Intent	Action	Key
play	PLAY	media_play	playpause
stop	PAUSE	media_pause	playpause
next	NEXT	media_next_track	nexttrack
previous	PREVIOUS	media_previous_track	prevtrack
mute	MUTE	audio_mute	volumemute
increase	VOLUME_UP	volume_increase	volumeup
decrease	VOLUME_DOWN	volume_decrease	volumedown

Stop means the PAUSE intent here. Host MUTE is distinct from microphone mute. Resume is not a separate prototype keyword. Volume commands may send several presses; Chiranthan records the exact implementation count before acceptance. Negation such as do not play and text containing multiple supported keywords are known limitations until Harsh and the team approve a policy. No natural-language understanding is claimed.

6 Resource and data lifecycle

Five seconds produces 80,000 samples and 160,000 PCM bytes. ESP32 capture/transport uses small reusable chunks; peak memory must be measured or reviewed on the actual firmware. The desktop read_exact and WAV-writing strategy must be inspected to determine its actual memory use; embedded streaming does not prove the laptop also streams without buffering.

recordings/recording.wav is generated at runtime. The guide does not establish deletion on success or failure. The no-retained-audio requirement remains unresolved until an explicit retention policy and cleanup behavior are implemented and tested. Generated speech recordings, model caches, environments and secrets should be excluded from source control. A file path being printed is not proof of cleanup.

Faster-Whisper model loading can dominate startup. Separate first-run installation, warm cached startup, capture, transcription and dispatch timings. The model is not running on the ESP32. Do not infer target memory fit or battery lifetime from this desktop architecture.

7 Errors and security

Failure	Required containment and diagnostic expectation
Missing or invalid port argument	Show useful usage/configuration error and do not start unintended recording.
Port busy or disconnected	Report acquisition failure, close acquired resources and send no media key.
Bad AUD0, length, truncated payload or END0	Reject the packet; do not transcribe a stale or partially written WAV as this run's recording.
Model unavailable or transcription fails	Report the failed stage; do not dispatch an action from old text.
Empty or unsupported text	UNKNOWN/unknown path produces no media key.
Invalid action or OS dispatch failure	Do not claim successful execution; record an appropriate diagnostic.

Finite timeout and serial recovery mechanisms are implementation tasks, not guaranteed current behavior. No TLS speech transport or cloud credentials are required. Network inspection with model cached verifies local inference separately from the media player's traffic. Console transcripts are visible to the local operator; this differs from the original command-ID-only logging policy and must remain an explicit deviation. Avoid sensitive information in committed logs.

8 Design decisions and deviations

Earlier baseline	Revised prototype decision	Consequence
On-board tinyML recognition	Local Windows Faster-Whisper	Mandatory laptop; revise F-004 and recognition tests.
BLE HID transport	USB serial plus PyAutoGUI	Pairing, BLE retry and associated security tests deferred.
Wake-driven continuous operation	One serial-requested recording per invocation	F-001/F-003 and idle-listening power tests deferred.
Resume in seven commands	Mute replaces Resume; stop maps to PAUSE	SRS vocabulary change requires instructor review.
Discrete state selection	playpause toggle for PLAY and PAUSE	Desired playback state cannot be guaranteed.
No retained audio	Runtime WAV written on laptop	SR-004 unresolved until retention decision and tests.
<= 3 second end-to-end latency	Fixed five-second capture plus processing	Original limit deferred; replacement threshold not selected.

Earlier baseline	Revised prototype decision	Consequence
Device maintenance services	Manual firmware upload and CLI setup	Wi-Fi portal, reset, LED/buzzer, OTA and signing deferred.

These are proposed baseline changes requested through the revised SRS, not retrospective acceptance of the original requirements. Preserve original IDs, submitted documents and completed modules. All revised meanings must be reflected in the STP and backlog before final course acceptance.

9 Testability and requirement allocation

Dheekshith's acquisition and serial components cover active F-002/F-012 and audio retention work under SR-004. Harsh's recognition/classification covers F-004/F-005/F-006, NF-002 and SR-001. Chiranthan's mapper/controller and console reporting cover F-007/F-008/F-010/F-014. Dhanush's CLI and orchestration cover F-011 and end-to-end integration. NF-005 checks compatible module replacement. NF-001/NF-003 require measurements and approval of revised performance/setup interpretation. The STP provides a row for every one of the 25 existing requirement IDs, including deferred cases.

10 Deployment and configuration management

The guide presents VoiceAssistant_ESP32.ino beside six Python modules and a generated recordings directory. This is a logical module inventory, not an instruction to restructure the repository. Record actual repository paths in release evidence. Select the actual Arduino board and port, verify/upload firmware and close Serial Monitor. Install desktop dependencies in an isolated environment, cache the model and run with the chosen COM port.

Use one story branch per change, retain module ownership and request independent review. Record the exact source commit, firmware build identity, Python/dependency versions and model identity in test records. Do not imply that an older portable C++ CI suite validates new serial/model/Windows behavior. No fresh build or deployment was performed to author this document.

11 Risks and pending design decisions

Item	Responsible member	Evidence or decision needed
Serial robustness and stale WAV prevention	Dheekshith with Dhanush	Actual timeout/readiness/recovery policy and malformed-packet tests.
Negation and multiple keywords	Harsh	Approved deterministic handling policy and regression cases.
Playback state and volume count	Chiranthan	Document current press count; retain toggle limitation or approve a state-aware extension.
Audio retention and console disclosure	Dheekshith with all owners	Approved deletion/retention policy and success/error-path evidence.
Timing, setup and accuracy	Assigned SRS owners	Measured trials, controlled environment and reviewed thresholds.
Course scope approval	Dhanush with all members	Instructor review of the change register and consistent SRS/SAD/STP.

12 Appendices and approval record

PCM is pulse-code modulation; WAV is the audio container; I2S is the digital microphone bus; intent is a classified command; action is the internal execution identifier; RTM is the requirement-to-test mapping. Pyserial supplies serial access, wave writes the WAV container, struct decodes length and PyAutoGUI generates host input events.

References: supplied complete project guide; updated SRS v1.3 dated 11 November 2026; updated STP v1.3; historical 7 November test-report bundle and supplied prototype demo. Historical results have their original scope and dates. Document owner coordination: Dhanush; technical reconciliation: Harsh; module content: all four. No team/instructor approval signature or independent test result is prefilled.